End of Studies Project Presentation
In regard of Obtaining The : National Diploma of Software Engineering

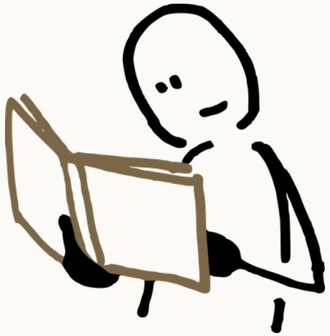
Design & Implementation of a Confidential Token Renewal Service

Evaluated By:
Mr. Hatem Sandid
Mr. Oussema Ben Youssef

Supervised By:
Prof. Dr. Christof Fetzer
Mr. Julius Wenzel
Mr. Yassine Boukhris

PRESENTED BY: MERIAM SOUIAI

9 SEP 2026



Plan

OF THE PRESENTATION

01 **General Context**

02 **Requirements Specification**

03 **Design & Architecture**

04 **Policy & Threat Model**

05 **Realisation**

06 **Conclusion & Perspective**



01

General Context

Host Organization and Program

KOSPIE Program

Combined Study and Practice Stays for Engineers from Developing Countries Tunisia

Funded By the DAAD Deutscher Akademischer Austausch Dienst German Academic Exchange Service with support from the BMZ Bundesministerium für wirtschaftliche Zusammenarbeit und Entwicklung .

Host Instituation: Technische Universität Dresden (TU Dresden)

One of Germany's major universities of technology and research institutions



- **Founded in 1828**
- **University of Excellence since 2012**
- **17** Faculties across 5 Schools
- 8,400 employees
- of whom 5,500 are academic staff **(38 % women, 25 % international researchers)**
- of whom 3,600 are funded by third parties

The Chair of Systems Engineering

Part of the **Institute of Systems Architecture** within the **Faculty of Computer Science** at **TU Dresden** was founded in April 2004.

The group addresses various research questions in the area of

- Security and Reliability of Distributed Systems and Cloud Infrastructures
- Fault-tolerant Computing in WANs
- Cost-effective Resilience

Recently, the group has conducted research on the use of **Intel SGX technology** to **provide security solutions for cloud-based applications.**



Faculty of Computer Science, one of the largest education and research institutions in Saxony

Technische Universität Dresden



The Systems Engineering (SE) group **headed by Prof. Christof Fetzer**

Motivation & Problem Statement

IN DEVOPS AND DEVSECOPS, WE SPEND A LOT OF TIME SECURING CODE, SCANNING DEPENDENCIES, HARDENING CONTAINERS, AND PROTECTING CLOUD INFRASTRUCTURE.

one ✕ **can create**  **IS POOR SECRETS MANAGEMENT**

A secret can be an any sensitive value used by an application or pipeline.

- API key
- database password
- cloud access token
- container registry credentials
- Gitlab/Github token

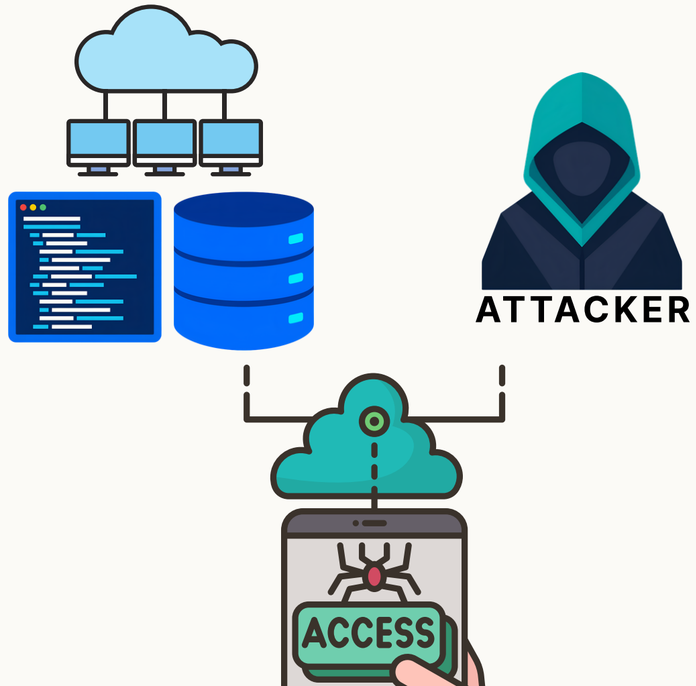


In a CI/CD environment, secrets are everywhere.

Pipelines use secrets to

- pull code
- build images
- push artifacts
- deploy infrastructure
- connect to cloud platforms
- run tests
- access Kubernetes clusters
- trigger deployments
- ...

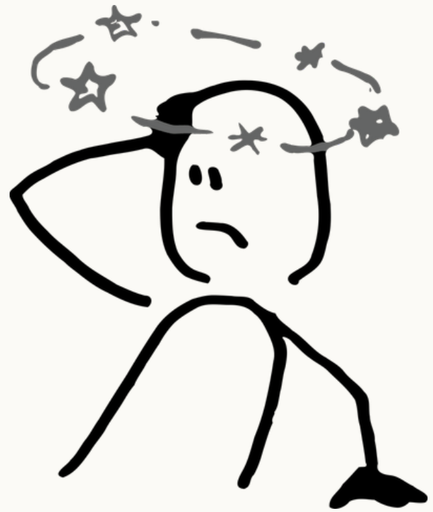
THIS MAKES CI/CD PIPELINES POWERFUL, BUT IT ALSO MAKES THEM SENSITIVE.



many of these tokens expire or need to be rotated regularly

unoticed → outages

- **Manual Process** → Administrators create or extend a token, store it as a CI/CD secret, and update the affected workflows
- Storing sensitive tokens directly in CI variables = **trust CI/CD Provider + infrastructure**: every job/runner that sees them
- **No central dashboard** overview of what tokens / where / when / how regenerated





“How can we design a system in which CI jobs can obtain fresh, short-lived access tokens, while the actual long-lived secrets remain hidden from the CI/CD platform and are only accessible to authorized code running on approved machines?”

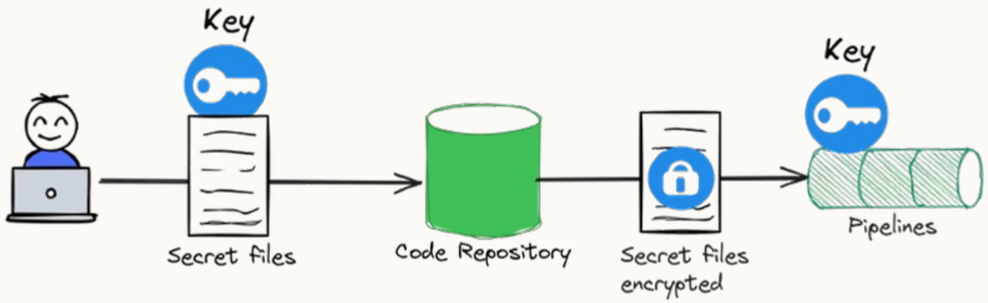
Research Question

Analysis of the existing solutions

ANSIBLE

+ protects secrets mainly through encryption

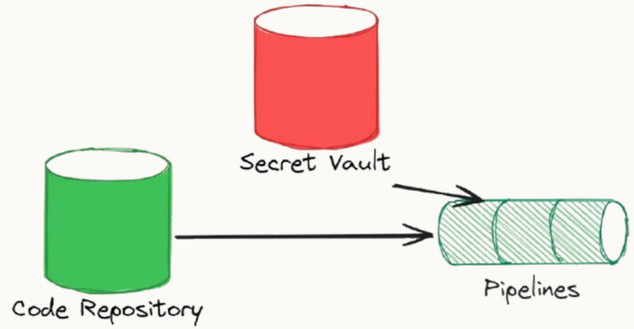
- high risk of compromised encryption key



HashiCorp Vault

+ third-party tools to secret management

- no secret rotation and complex to maintain



BOTH RELY ON LONG-LIVED CREDENTIALS

Goals



1

Automatic renewal

regenerate tokens via provider APIs when possible

2

Central Dashboard

inventory, expiry, usage, renewal status/history

3

Confidential secrets

long-lived credentials never stored as CI secrets

4

Runner-bound access

real tokens only generated or released to code on authorized machines

5

Auditable usage

log who/what got a token (workflow, runner, policy, outcome)

6

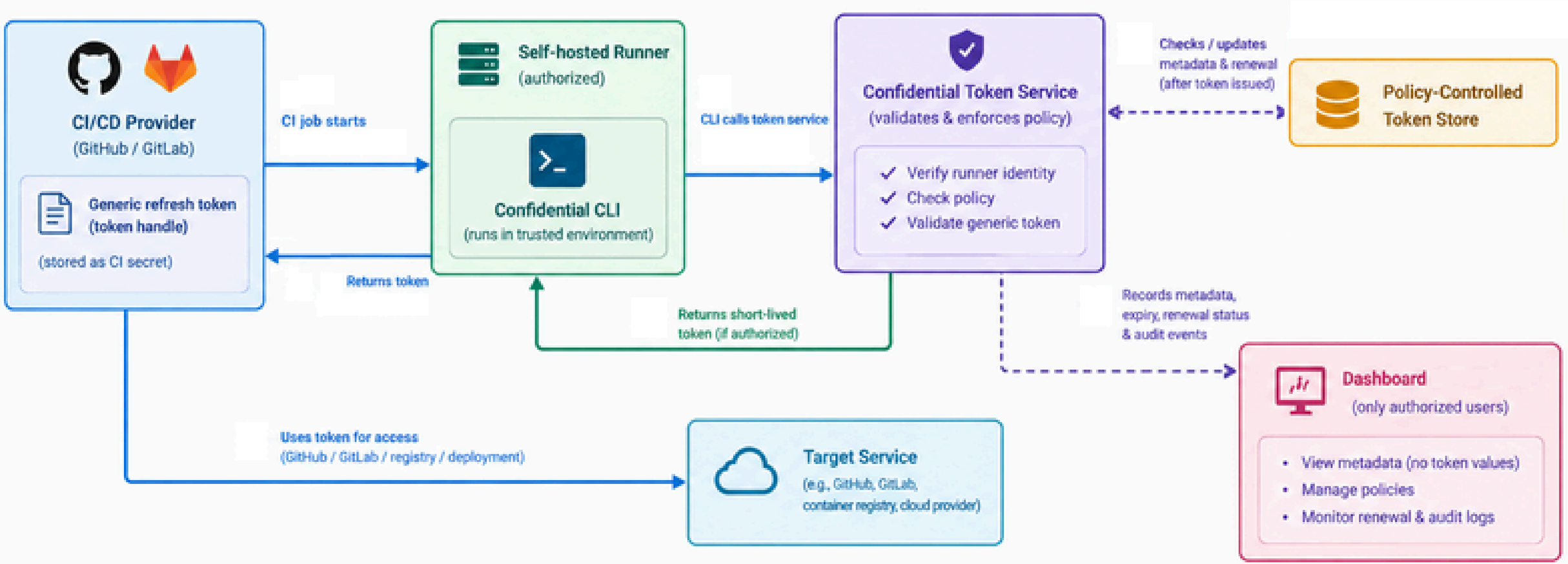
Separation of visibility and control

operators see metadata, expiry status but not token values

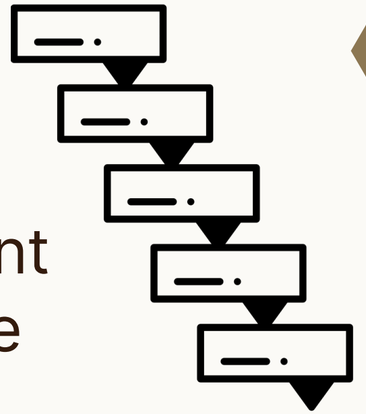
Proposed Solution

CTRS - Confidential Token Renewal Service

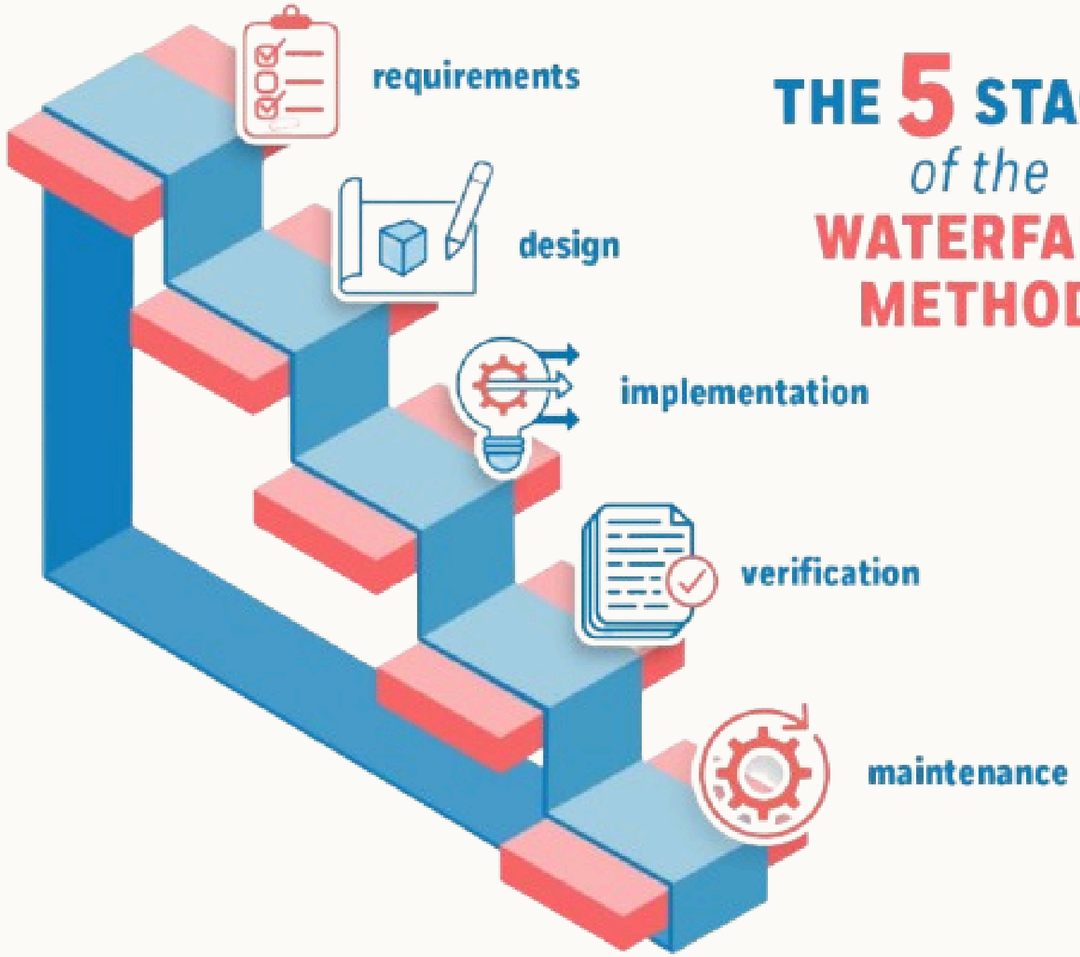
The suggested solution consists of three main components: a dashboard service, a confidential token-conversion CLI, and a policy-controlled token store.



Waterfall Methodology



The waterfall methodology is a linear, sequential project management approach where each phase must be completed before the next one begins.





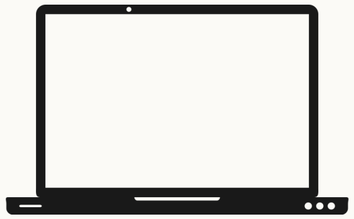
02

Requirements Specification

System Actors



Operator



Self-hosted Runner



Scheduler

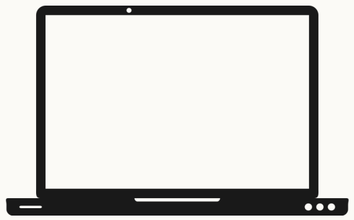
Functional Requirements



- Register a real token into the vault and get back a handle
- Define policies
- Rotate and Revoke tokens without changing the CI handle
- View Central Dashboard and audit logs



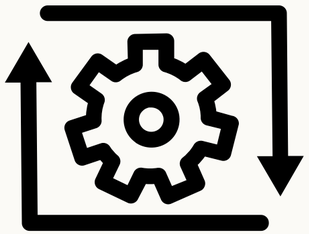
- Periodically check token expiration dates
- Trigger automatic renewal before a token expires



- Send an exchange request
- Receive the short-lived token and pass it to the CI job only

Non-Functional Requirements

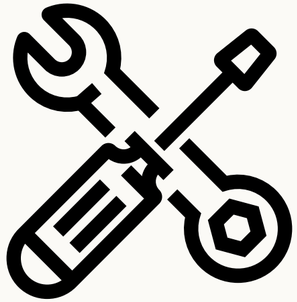
Reliability



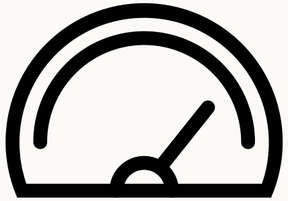
Security



Maintainability



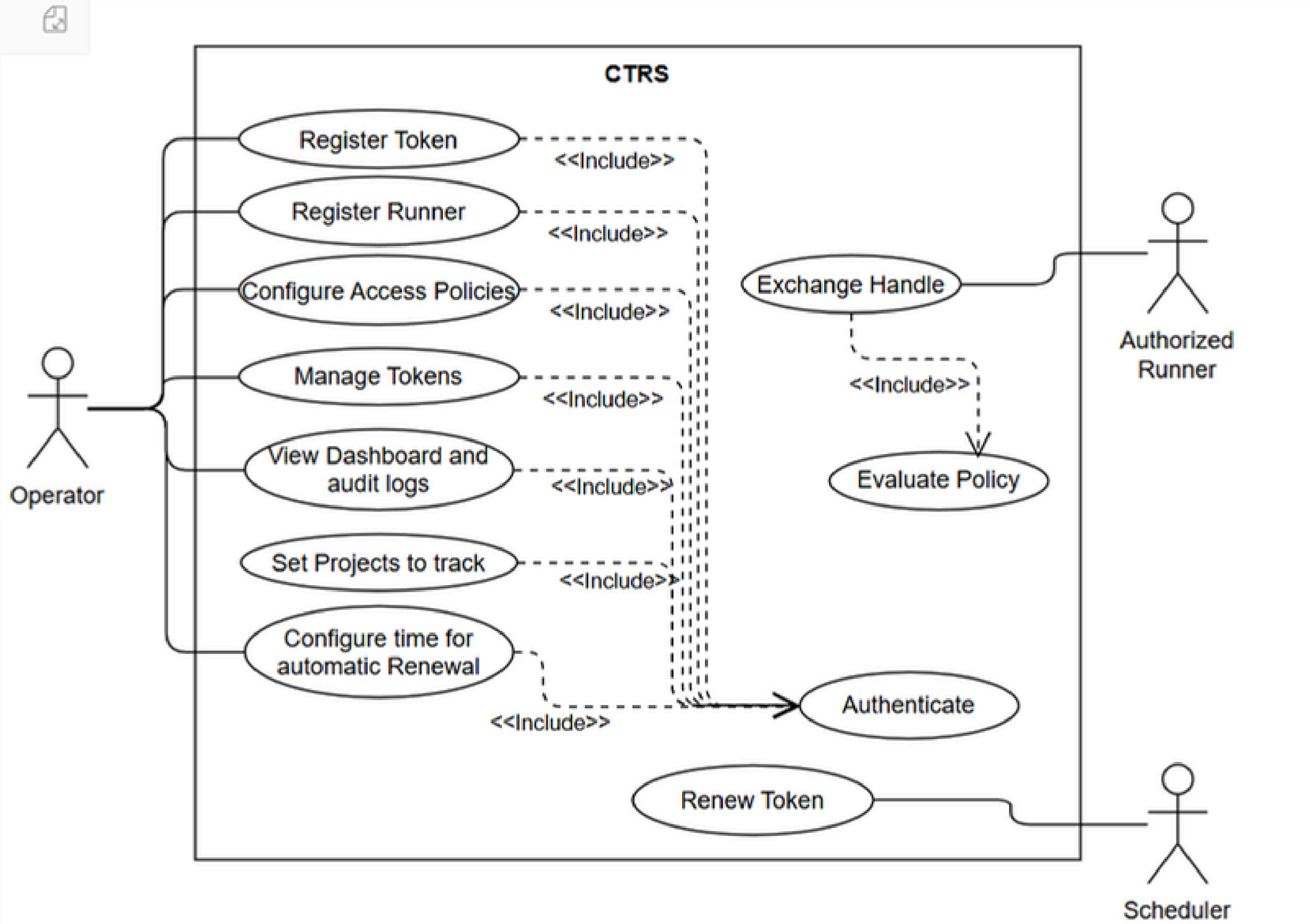
Performance



Ergonomics



USE CASE Diagram





03

Design & Architecture

Architectural pattern

3 TIER

Presentation tier (Client)



Application tier (Server)

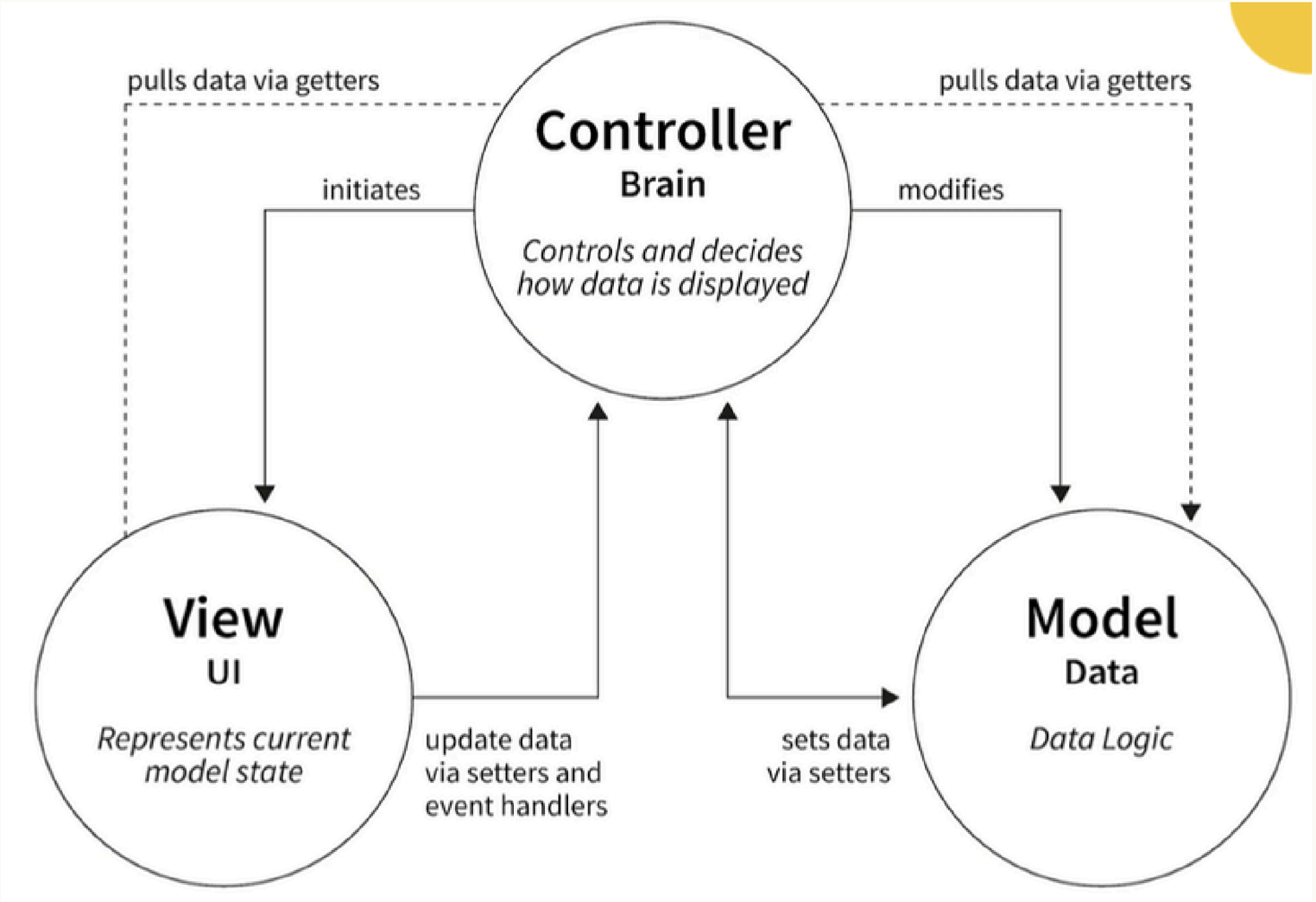


Data tier



Design Pattern

MVC

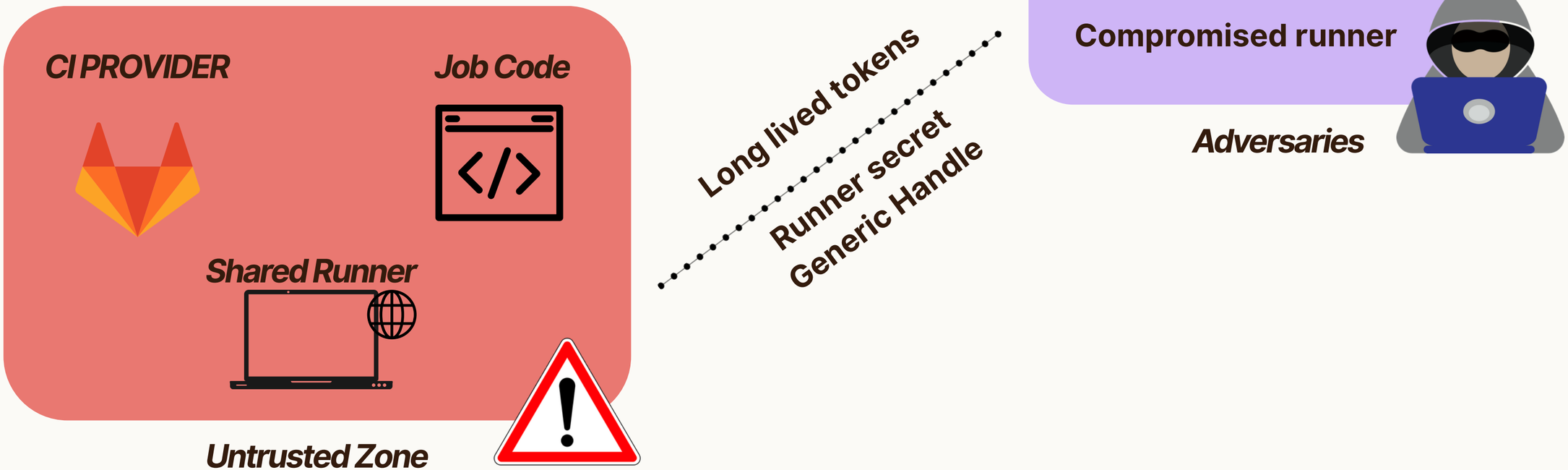




04

Policy & Threat Model

Threat Model



Attacker	What they can do	Solution Used
External Attacker	Call the API from the internet and get real token	TLS + runner-secret check
Compromised CI job	Read any CI variable injected into the job	CI stores only handle not real token; handle alone cannot call GitLab
Malicious repo contributor	Modify the workflow YAML	Default-deny policies pinned to repo + branch + job/workflow
Compromised runner	Run arbitrary code on the trusted machine	Runner must be pre-registered with secret; audits record exchanges but improve with TPM/attestation
Malicious dashboard user	Access the operator UI and all API endpoints	Dashboard/API return metadata only; no secret in plain text

Policy Model

- Each policy rule has a **priority** : higher number is evaluated first
- Each rule targets one specific **token** in the vault
- Each rule defines conditions that must all match:
  - Which **runner** is allowed
  - Which **repository** the job must come from
  - Which **workflow / job name** is permitted
  - Which **branch** the job must run on
  - Maximum **token lifetime** the issued token can have

priority	`100`
token	`tok-registry-deploy`
runner	`ctrs-demo`
repository	`meso598i/token-test`
workflow	`ctrs_exchange_and_api`
branch	`main`
max lifetime	`3600 seconds (1 hour)`



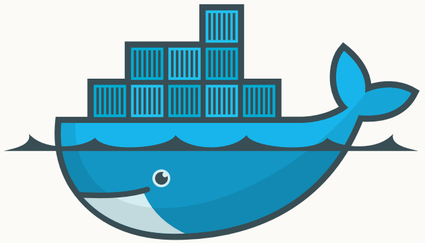
05

Realisation

Tools



git



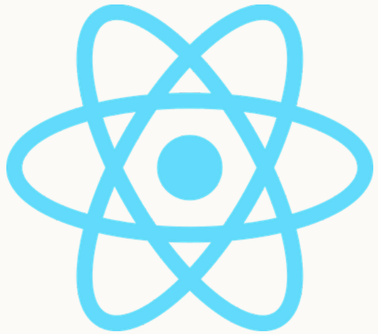








Technologies









“Confstr” - The Confidential CLI

TURNS A HANDLE INTO A REAL TOKEN AND ONLY ON A TRUSTED RUNNER

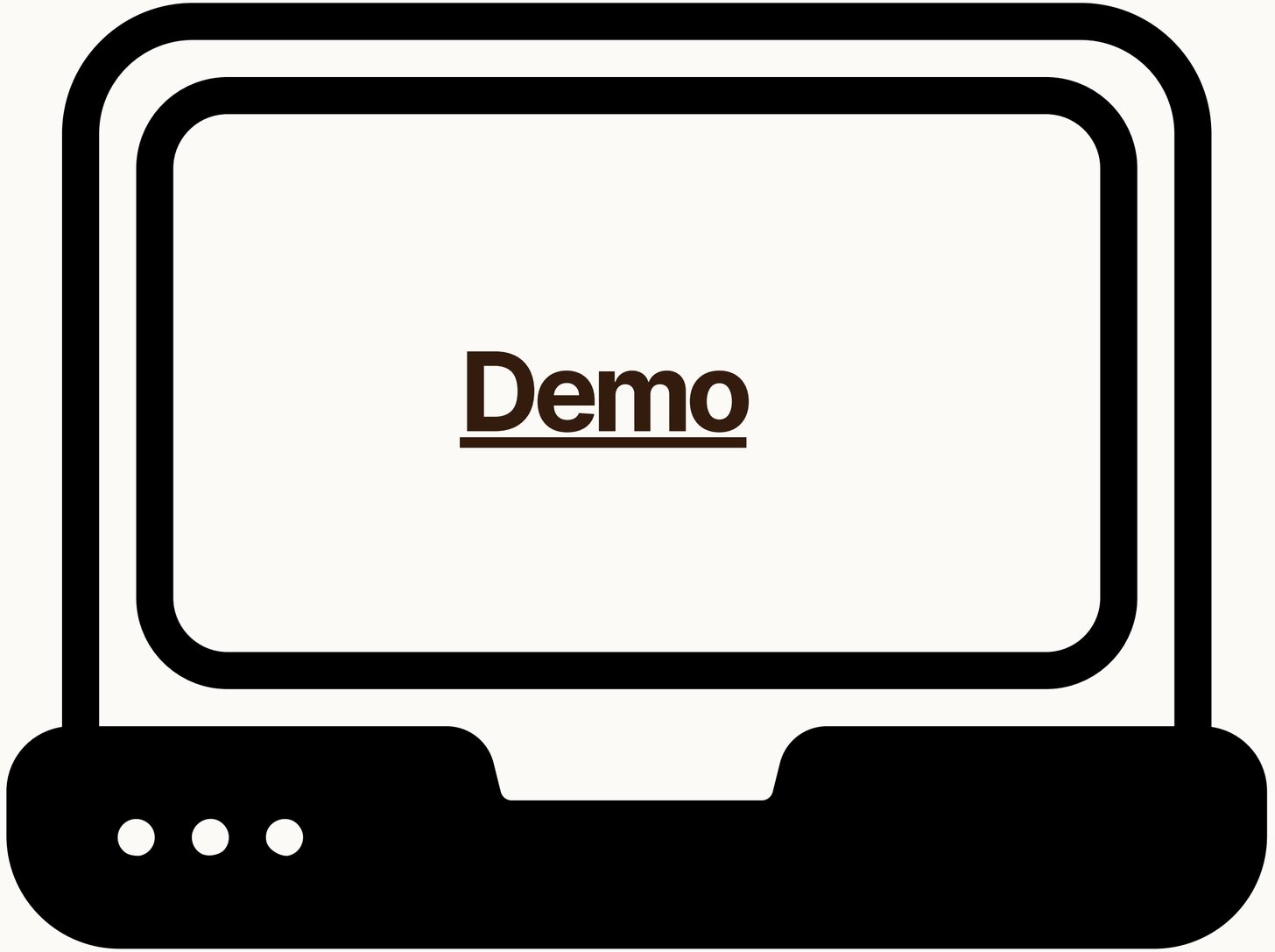
```
meria@DESKTOP-R2A95GQ:/mnt/c/Users/meria/OneDrive/Desktop/pfe$ confstr --help
Usage: confstr [OPTIONS] COMMAND [ARGS]...


  confstr — exchange generic token handles for short-lived access tokens.

Options:
  --help  Show this message and exit.

Commands:
  exchange       Exchange a generic handle for a short-lived real token.
  prove-host     Prove /etc/ctrs/runner-secret matches the runner...
  verify-runner  Check whether this machine appears to be an authorized...
```

`confstr exchange`	Handle → short-lived access token
`confstr prove-host`	Check runner secret matches CTRS registration
`confstr verify-runner`	Detect runner environment / CI context


Demo



06

Conclusion & Perspectives

Evaluation

- Dashboard gives central view: tokens, expiry, renewals, audit
- CI stores handle only (no PAT in CI variables)
- Token exchange allowed only for approved runner + matching policy
- Manual + scheduled token rotation + Revoke
- Notification and automatic alerting when renewal fails or suspicious behavior appears..

Current limitation and Future Work

- Trust is still based on a secret file on the runner machine.

Replace shared runner secrets with remote attestation (TPM / TEE)

- Extend the architecture beyond GitLab CI/CD environment so it also supports other CI/CD and cloud providers



Conclusion

Built and tested a working prototype for for Gitlab CI/CD that manages token renewal system with an overall Admin dashboard for tracking.
CI jobs no longer need to store the real long-lived GitLab token in CI variables. Jobs receive access only when they run on an approved runner and match the policy rules.

My Experience

Worked on confidential computing research project

Gained a genuine research experience

Improved both technical skills & Softskills



End of Studies Project Presentation
In regard of Obtaining The : National Diploma of Software Engineering



Thank you !
Welcoming your questions

PRESENTED BY MERIAM SOUIAI

End of Studies Project Presentation
In regard of Obtaining The : National Diploma of Software Engineering

Design & Implementation of a Confidential Token Renewal Service

Evaluated By:
Mr. Hatem Sandid
Mr. Oussema Ben Youssef

Supervised By:
Prof. Dr. Christof Fetzer
Mr. Julius Wenzel
Mr. Yassine Boukhris

PRESENTED BY: MERIAM SOUIAI

9 SEP 2026